

Software Engineering Reviewer

Architectural Design, Software Design & Implementation,
Testing & Evolution

Made by James Adrian B. Castro

A comprehensive reviewer covering:

- Architectural Design
- Software Design & Implementation
 - Software Testing
 - Software Evolution

PART 1

Architectural Design

1. What is Architectural Design?

Architectural design involves planning and creating buildings that emphasize both aesthetics and functionality. In software engineering, it serves as the comprehensive high-level blueprint that outlines the fundamental structure of a software system, detailing its main components, how they interact, and the mechanisms through which they communicate to achieve the system's objectives.

Importance of Architectural Design

- Serves as a fundamental blueprint guiding software development.
- Emphasizes scalability to handle growth, maintainability for ease of updates, and adaptability to evolving requirements.
- Ensures the creation of reliable, efficient, and robust software solutions.

Common Architectural Styles / Patterns

- Layered Architecture: Presentation, business, persistence layers
- Client-Server: Clients request services from a server
- MVC / MVVM: Separates UI, logic, and data
- Microservices: Small, independently deployable services
- Service-Oriented Architecture (SOA): Services communicate over a network
- Event-driven: Components react to events
- Pipe-and-Filter: Data flows through processing steps
- Repository / Blackboard: Shared data store accessed by components
- Peer-to-Peer: Decentralized nodes communicate directly
- Serverless: Cloud functions managed by a provider

2. The Architectural Design Process

1. Understanding requirements and constraints (Identify ASRs and constraints from stakeholders).
2. Choosing a structure (Choose architectural style(s) based on quality goals).
3. Defining components, interfaces, and communication (Decompose the system; define responsibilities, APIs, events, databases).
4. Deployment, documentation, and evaluation (Allocate to infrastructure; document views/ADRs; check against quality attributes).
5. Iteration (Start from #1 again; architecture is rarely final in one pass).

3. Evaluation Methods in Architectural Design

- ATAM: Architecture Tradeoff Analysis Method
- SAAM: Software Architecture Analysis Method

- CBAM: Cost-Benefit Analysis Method
- Risk Analysis: Identifying architectural risks early on.

Example Scenario (E-commerce System)

An architecture might use:

- Microservices for orders, payments, inventory, users
- API Gateway for client requests
- Event bus for order/payment events
- Database per service for independence
- CDN and caching for performance
- Load balancer for availability

PART 2

Software Design & Implementation

1. Core Concepts

- **Software Design:** Executing the design by turning concepts into functional software through installation, configuration, customization, and system integration.
- **Software Implementation:** Planning the system by converting requirements into a detailed blueprint, defining the architecture, and breaking complex functions into manageable modules.
- **Object-Oriented Design (OOD):** Modeling systems using real-world concepts like classes, objects, encapsulation, inheritance, and polymorphism to build modular, adaptable, and easily understandable architectures.

2. Defining System Context and Operational Boundaries

This defines what the system can and cannot do. It identifies the users, databases, and other systems that interact with the system.

- **System Context:** Identifying what the system covers and what is outside its responsibility.
- **External Actors:** Identifying users, other systems, and databases that interact with the system.
- **Object Roles:** Grouping objects into boundary, entity, and control roles based on their responsibilities.
- **CRC Cards:** Helping identify a class's responsibilities and relationships with other classes.

3. Architectural Design and Interface Specifications

This explains how the system is organized and how its parts communicate. These designs serve as a guide for implementation, where developers turn the planned components and interfaces into actual working code.

- **Architectural Design:** Planning the system's major components and how they are connected.
- **Interface Specification:** Defining the inputs, outputs, and operations between system components.
- **Design by Contract:** Making each component's responsibilities and requirements clear.
- **UML Diagrams:** Using class diagrams to show structure and sequence diagrams to show interactions.

4. Design Modeling Dimensions

- **Static Models (System Structure Design):** Focus on system structure and relationships (e.g., class diagrams, ER diagrams).
- **Dynamic Models (System Interactions & Flow):** Capture system interactions, workflows, and behavior over time (e.g., sequence diagrams, activity diagrams).
- **Design & Blueprints:** Provide architectural and detailed design specifications for implementation.

5. Implementation Considerations

- **Key Implementation:** Critical architectural and operational decisions developers make to translate conceptual models into executable source code.
- **Component Reuse & COTS:** Using existing libraries, frameworks, and Commercial Off-The-Shelf systems to

reduce development costs and accelerate time-to-market.

- Open-Source Compliance: Managing licensing agreements (MIT, GPL, Apache) to ensure legal compliance and avoid intellectual property infringement.
- Configuration Management: Tracks and manages code, docs, and setup files across the software lifecycle. It ensures that changes across team members are controlled and system versions remain consistent.

6. Conclusion & Takeaways

Minimizing Bug Rework: Structured software design provides a clear blueprint that connects system requirements, architecture, components, and interfaces before implementation begins. By defining system boundaries, object responsibilities, models, and interfaces early, developers can identify problems sooner, reduce implementation bugs, and avoid unnecessary rework.

Core Takeaways: Thoughtful and structured design makes implementation more organized, reduces errors, simplifies maintenance, and helps create scalable and reliable software.

PART 3

Software Testing

1. What is Software Testing?

- A systematic process of running a program to uncover errors introduced during design and construction.
- It often takes more effort than any other step in the development process.
- Testing alone can account for roughly 40% of total project cost.
- A solid testing strategy maps out test planning, case design, execution, and evaluation.

Verification vs Validation

Verification	Validation
'Are we building the product right?'	'Are we building the right product?'
Correctly implements the function	Traceable to customer requirements

2. Important Software Testing Concepts

- Test Case: Contains conditions, inputs, steps, and expected results to check a specific function.
- Test Scenario: Describes what part or functionality of the system needs to be tested.
- Test Data: The information or values used during testing (e.g., valid/invalid username).
- Bug or Defect: An error in the software that causes an incorrect or unexpected result.
- Expected Result: What should happen when a particular test is performed.
- Actual Result: What really happens when the software is tested.

3. Levels of Software Testing

Testing Strategies for Conventional Software can be viewed as a spiral consisting of four levels:

1. Unit Testing: Focuses on testing each individual unit or component at the source code level. Includes boundary testing (software commonly fails at its boundaries).
2. Integration Testing: Focuses on how units work together. Top-down (starts from main control module), Bottom-up (starts with lowest-level modules), Sandwich strategy (hybrid).
3. System Testing: Evaluates a fully integrated application as a whole. Recovery Testing (MTTR), Security Testing, Stress Testing, Performance Testing.
4. Validation Testing: Evaluates the constructed software against established requirements. Validation Test Criteria (SRS), Configuration Review, User Feedback Loops.

User Acceptance Test (UAT)

Ensures the software meets business needs and user expectations.

Requirement Analysis -> UAT Test Plan Creation -> Identify Test Scenarios -> Create UAT Test Cases -> Prepare Test Data -> Test Run -> Confirm Business Objectives

Alpha and Beta Test

Phase	Location and Environment	Operational Setup
Alpha Testing	Developer's site (Controlled)	Performed by end-users in a simulated setting
Beta Testing	End-user's site (Uncontrolled)	A 'live' application deployment where users report real-world problems

4. Testing Tactics

- Black box testing: Examines some fundamental aspect of a system; tests whether each function of the product is fully operational.
- White box testing: Examines the internal operations of a system and examines the procedural detail.

5. Manual vs Automated Testing

Manual Testing Pros	Automated Testing Pros
Cheap upfront	Faster execution
No tooling needed	Consistent results
Good for exploratory testing	Broader coverage
Flexible for changing requirements	Cost-effective over time
Manual Testing Cons	Automated Testing Cons
Doesn't scale	Upfront investment in tools
Results vary between testers	Not worth it for short-lived features
Slow for repetitive runs	

Where Automation Fits Best

New Build -> Smoke Test (quick filter) -> Regression Test (deep check) -> Release

- Smoke Testing: Quick check of build's basic stability.
- Regression Testing: Retests after changes to ensure nothing broke.

6. Specialized Testing

- API Testing: Checks API functionality, reliability, performance via requests/responses with no UI.
- Data-Driven Testing: Separates test data from test scripts for reuse and flexibility.
- Keyword-Driven Testing: Uses keywords/action words to define test steps.

7. The Art of Debugging

- Debugging occurs as a consequence of successful testing. It is an action that results in the removal of errors.
- It is the process of finding and fixing errors or bugs in a software program.
- Main difference: Testing finds the problem, while debugging finds its cause and fixes it.
- Two Outcomes: Cause will be found and corrected, OR cause will not be found.

Debugging Approaches

- Brute Force: Most common yet least efficient. Uses print statements, logs, or memory dumps. Inefficient searching through large amounts of output.
- Backtracking: Starts from where the error first appeared. Symptom-Based Start: Checks the exact line, function, or

module where the problem occurred. The code is then traced backward to find the cause.

- Cause Elimination: Based on binary partitioning. The developer creates a list of all possible causes and devises a specific 'cause hypothesis'. Data is used to prove or disprove the hypothesis.

PART 4

Software Evolution

1. What is Software Evolution?

Software evolution is the term used in software engineering to refer to the process of developing an initial version of the software and then repeatedly updating it to satisfy the user's needs.

Software evolution is an inevitable activity, as useful and successful software stimulates users to request new and improved features.

2. Software Evolution Core Processes

- Change Identification and Impact Analysis: Identify proposed changes and analyze their cost and impact on the system.
- Release Planning: Approve and prioritize which changes will be included in the next release.
- Change Implementation: Design, implement, and test the changes while using regression testing to ensure existing features still work properly.
- System Delivery: Deploy the updated version for users and identify new change requests for future updates.

Evolution Process Flow

Change requests -> Impact analysis -> Release planning -> Change implementation -> System release

Sub-processes: Fault repair, Platform adaptation, System enhancement

3. Lehman's Laws of Software Evolution

- 1. Continuing Change: Software must change to remain useful.
- 2. Increasing Complexity: Software becomes more complex over time.
- 3. Large Program Evolution: Evolution follows a stable pattern.
- 4. Organizational Stability: Development rate remains relatively constant.
- 5. Conservation of Familiarity: Changes should be gradual to maintain user familiarity.
- 6. Continuing Growth: Functionality must continue to improve to satisfy users.
- 7. Declining Quality: Quality declines if software is not adapted to changes.
- 8. Feedback System: Feedback helps improve the system.

4. Why is Software Evolution Important?

- Helps software adapt to changing user needs and business needs.
- Maintains user satisfaction.
- Fixes bugs and security issues.
- Improves performance.
- Keeps software compatible with new hardware and operating systems.
- Regular updates allow developers to add new features, reduce technical debt, support more users, extend the

software's lifespan, and keep it up to date with industry standards.